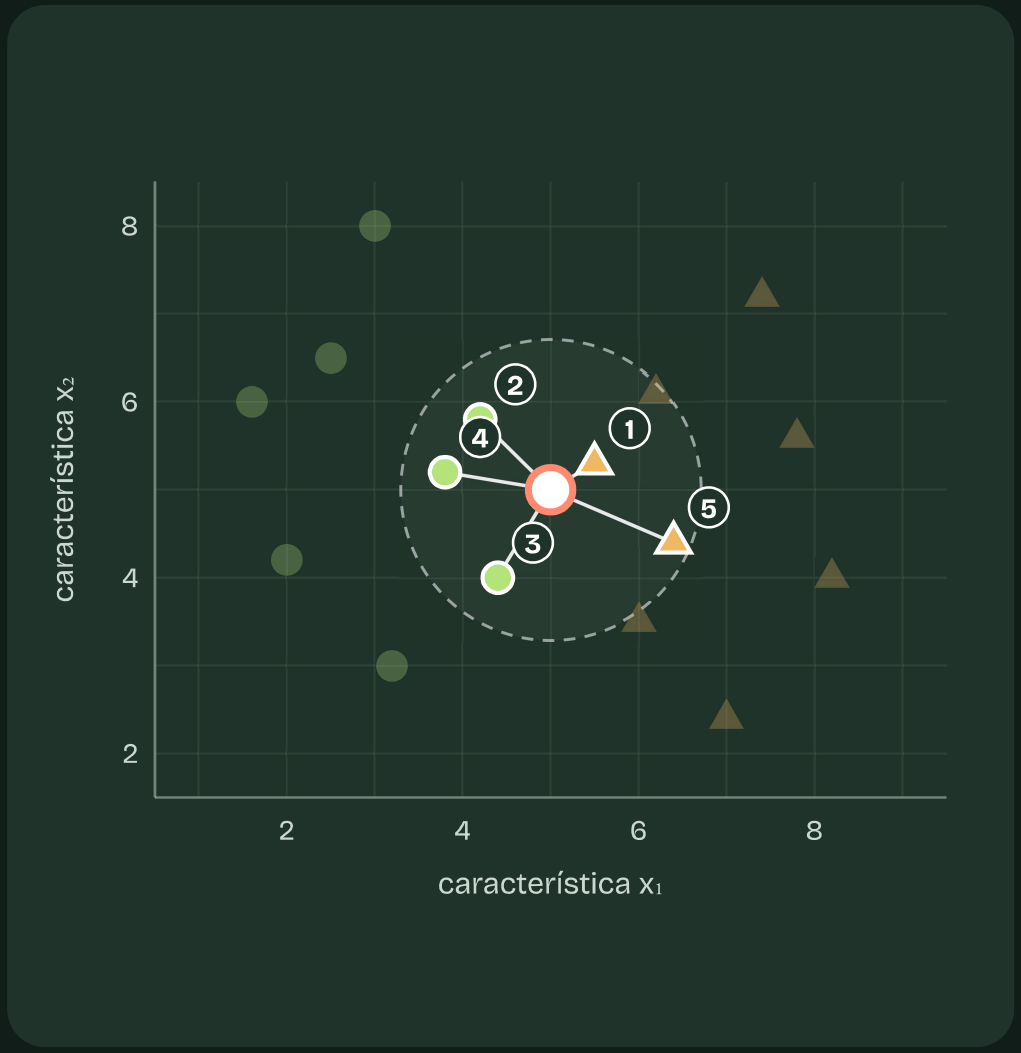
AULA 6.7 · MACHINE LEARNING

KNN do zero

K-Nearest Neighbors: classificar um exemplo pelos seus vizinhos mais próximos



DURAÇÃO
2 horas

FERRAMENTAS
Python · NumPy · Matplotlib

01 · OBJETIVOS

Objetivos da aula

Entender

- O significado de KNN e de K
- Como os vizinhos são escolhidos
- Votação majoritária e empates
- Por que escalonar os dados
- Broadcasting e vetorização

Calcular

- Distância euclidiana à mão
- Distância de Manhattan à mão
- Matriz de distâncias com NumPy
- Shapes de X_train e X_test
- Tempo e custo de memória

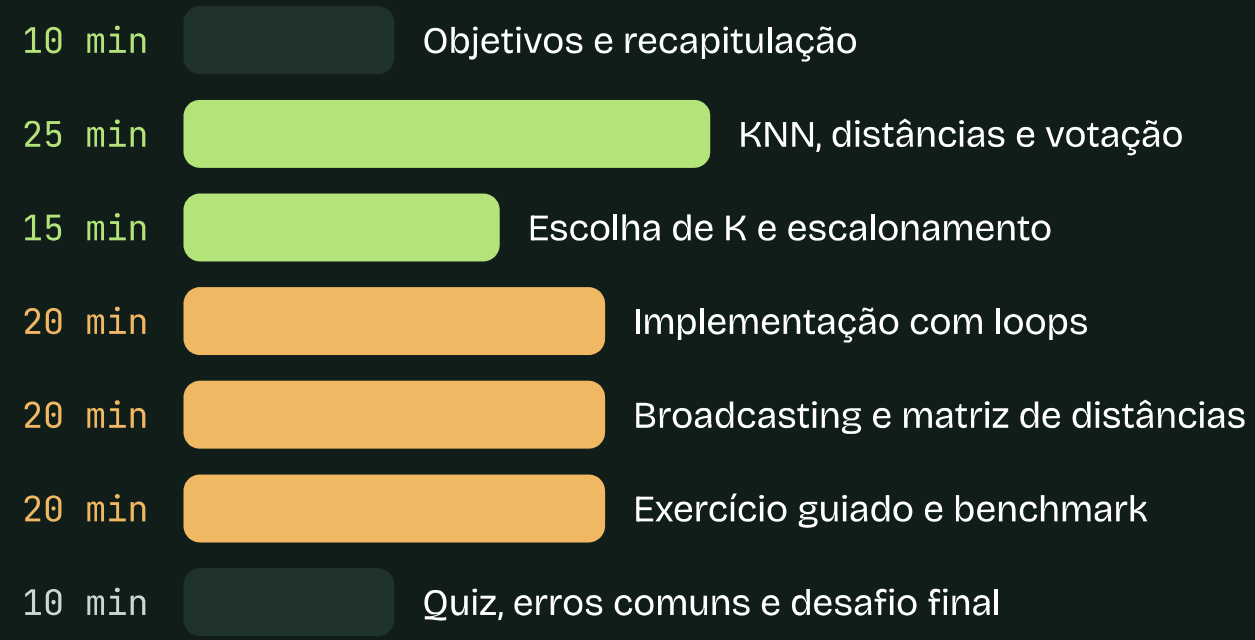
Implementar

- KNN com loops
- KNN com Broadcasting
- A classe KNNDoZero
- Benchmark com perf_counter
- Avaliação com acurácia

Meta: explicar como o KNN toma uma decisão, não apenas executar `KNeighborsClassifier`.

01 · OBJETIVOS

Roteiro das duas horas



02 · RECAPITULAÇÃO

Amostras, características e rótulos

amostra	peso (g)	diâm. (cm)	fruta
fruta 1	80	5,0	limão · 0
fruta 2	210	8,1	laranja · 1
fruta 3	95	5,6	limão · 0
fruta 4	180	7,4	laranja · 1

cada linha = 1 amostra

características → matriz X

rótulo → y

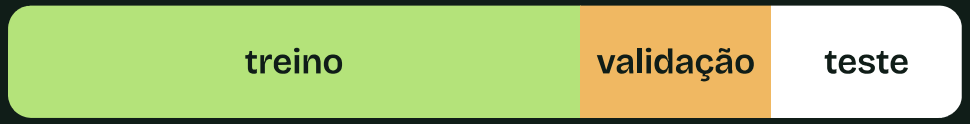
X → modelo → $\hat{y}$

$\hat{y}$ (y-chapéu) é a previsão. Comparamos $\hat{y}$ com o y verdadeiro para avaliar o classificador.

$$\text{acurácia} = \frac{\text{previsões corretas}}{\text{total de previsões}}$$

02 · RECAPITULAÇÃO

Treino, teste e regressão logística



Treino — o modelo aprende (ou memoriza) os exemplos.

Validação — escolhemos hiperparâmetros, como K. Pode ser feita com K-Fold (Aula 6.6).

Teste — usado só no final, para estimar o desempenho em dados novos.

Regressão logística aprende coeficientes

$$p = \sigma(w \cdot x + b)$$

x	características de uma amostra
w, b	coeficientes e intercepto aprendidos no treino
σ	sigmoide: transforma o resultado em probabilidade entre 0 e 1

Depois do treino, o modelo é resumido em w e b.

02 · RECAPITULAÇÃO

Uma nova pergunta

É possível classificar um exemplo observando quais outros exemplos são mais parecidos com ele?



- limão · classe 0
- laranja · classe 1
- fruta nova · classe desconhecida

03 · TEORIA

O que é KNN

K

quantos vizinhos consultamos

inteiro positivo escolhido por nós (ex.: 3, 5, 7)

Nearest

mais próximos

os de menor distância até o exemplo novo

Neighbors

vizinhos

exemplos de treino com rótulo conhecido

Hipótese do algoritmo: exemplos próximos no espaço das características tendem a ter o mesmo rótulo.

K do KNN ≠ K do K-Fold

`KNN`

K = número de vizinhos que votam na classe

`K-Fold (Aula 6.6)`

K = número de partes da validação cruzada

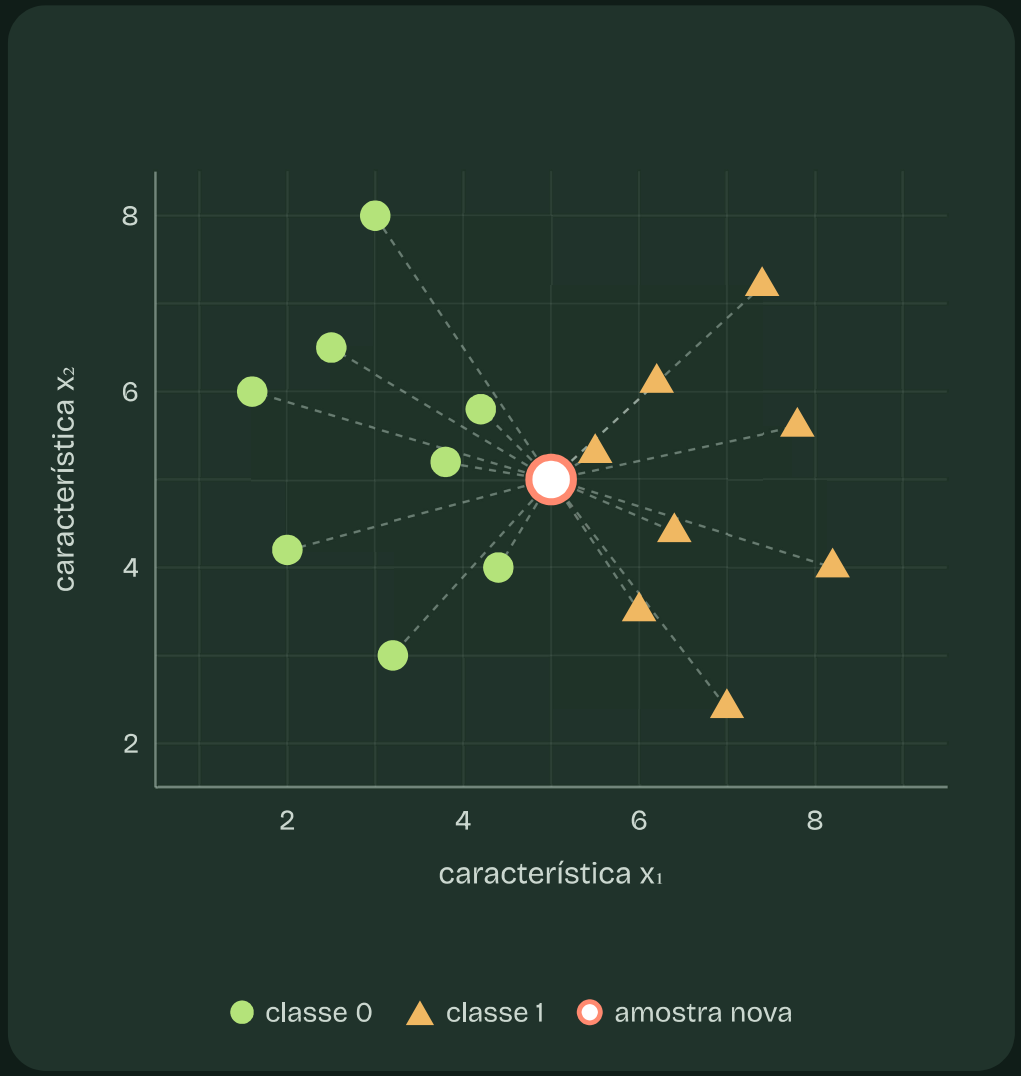
Os dois podem aparecer juntos: usar 5-Fold para escolher o K do KNN.

03 · TEORIA

Aprendizado baseado em instâncias

	Regressão logística	KNN
O que `fit()` faz	otimiza w e b	guarda `X_train` e `y_train`
O que fica na memória	poucos coeficientes	todo o conjunto de treino
Onde está o trabalho	no treino	na previsão: distâncias até todos os exemplos
Tipo de aprendizado	baseado em modelo	baseado em instâncias · lazy learning

Lazy (preguiçoso): o KNN adia todo o cálculo para o momento da previsão.

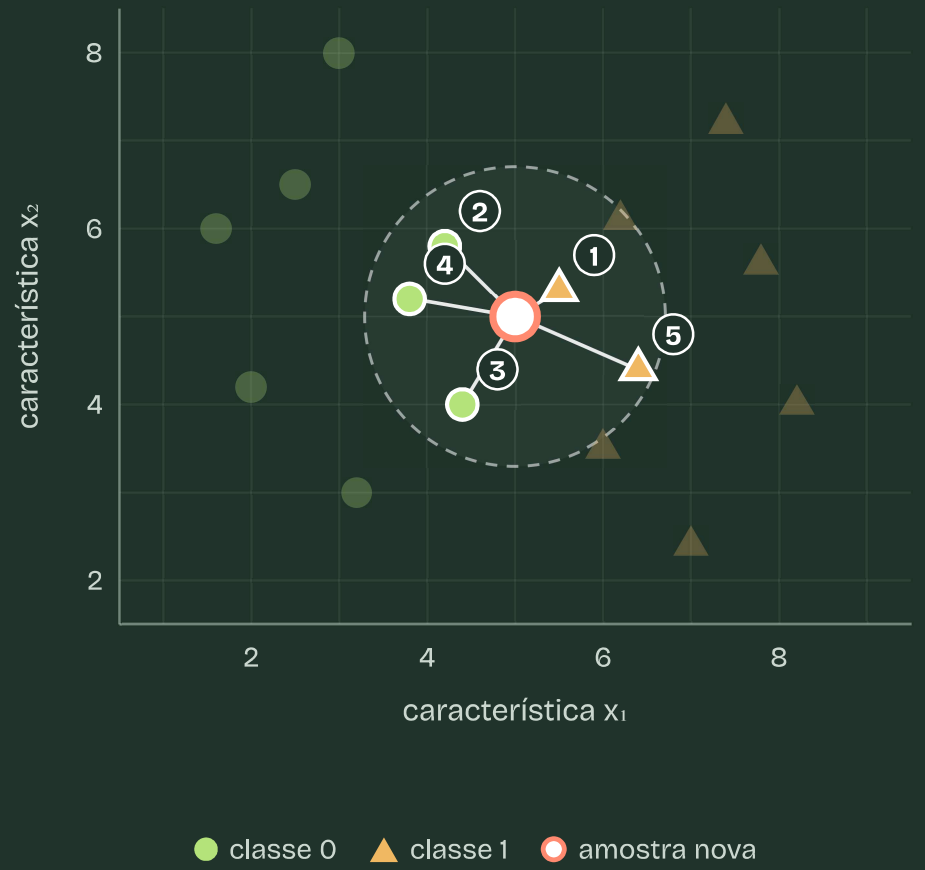


03 · TEORIA

KNN passo a passo

1. Receber a nova amostra (5, 5)
2. Calcular a distância até cada exemplo
3. Ordenar as distâncias
4. Separar os K mais próximos
5. Ler os rótulos dos vizinhos
6. Fazer a votação
7. Retornar a classe vencedora

16 exemplos de treino → 16 distâncias. Nenhum vizinho foi escolhido ainda.

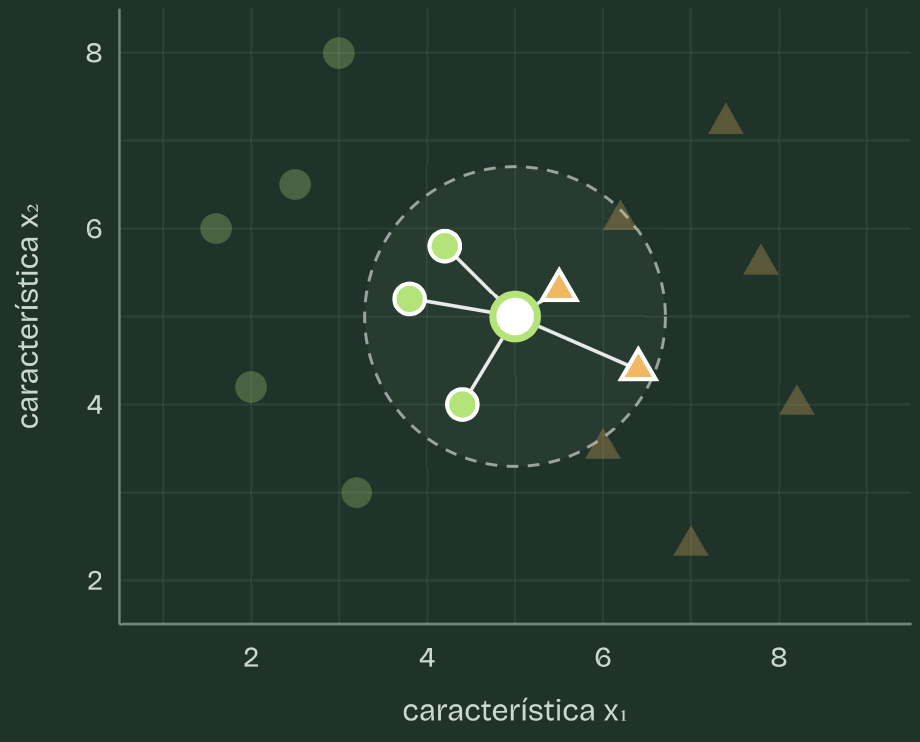


03 · TEORIA

KNN passo a passo

3 Ordenar 4 Separar os K = 5 mais próximos

posição	distância	classe
1	0,583	1 · triângulo
2	1,131	0 · círculo
3	1,166	0 · círculo
4	1,217	0 · círculo
5	1,523	1 · triângulo
6–16	≥ 1,628	descartados



03 · TEORIA

KNN passo a passo

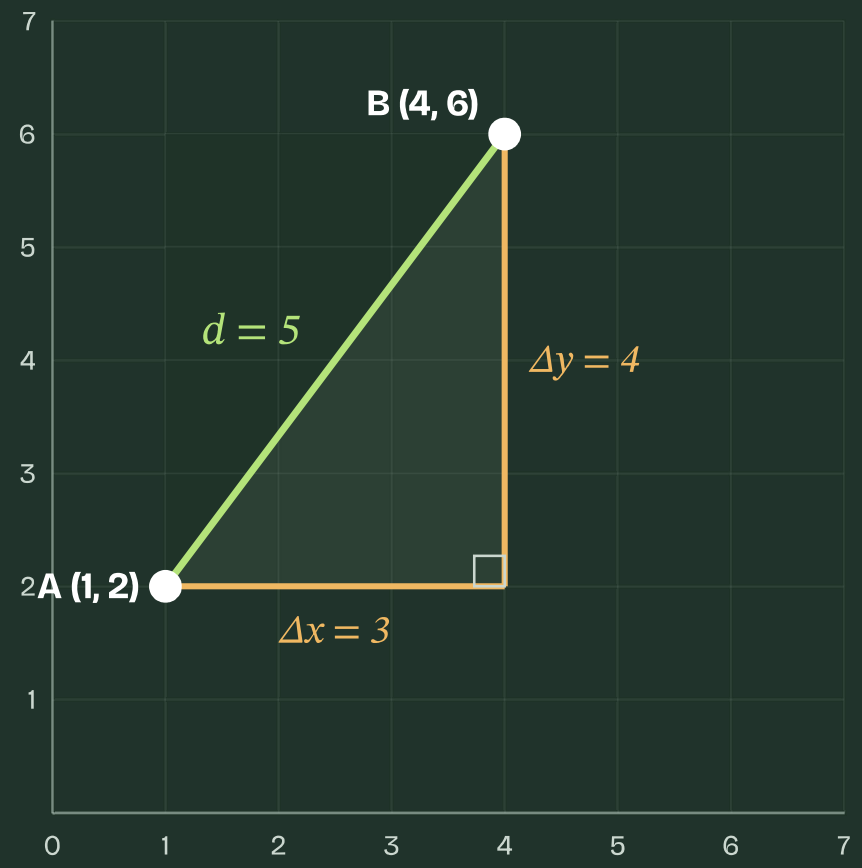
5 Rótulos 6 Votação 7 Resultado

1	0	0	0	1

classe 0 — 3

classe 1 — 2

Classe prevista **0**



03 · TEORIA · DISTÂNCIAS

Distância euclidiana

A distância em linha reta é a hipotenusa de um triângulo retângulo.

$$c^2 = a^2 + b^2$$ Pitágoras

$$d^2 = 3^2 + 4^2$$ catetos: 4 − 1 e 6 − 2

$$d^2 = 9 + 16 = 25$$ quadrados somados

$$d(\mathrm{A}, \mathrm{B}) = \sqrt{25} = 5$$ raiz quadrada

O quadrado elimina o sinal: $(1 - 4)^2 = (-3)^2 = 9$. Por isso $d(\mathrm{A}, \mathrm{B}) = d(\mathrm{B}, \mathrm{A})$.

Distância euclidiana: fórmula geral

$$d(x, y) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$

2 características (o plano cartesiano)

$$\sqrt{((x_1 - y_1)^2 + (x_2 - y_2)^2)}$$

n características: a mesma conta, com mais termos

$$\sqrt{((x_1 - y_1)^2 + \ldots + (x_n - y_n)^2)}$$

d	distância entre os dois pontos
x	primeiro ponto (vetor de características)
y	segundo ponto (aqui não é o rótulo)
i	índice que percorre as características
n	número de características
x_i, y_i	valor da característica i em cada ponto
$(-)^2$	diferença em uma característica, ao quadrado
Σ	somatório: soma os termos de i = 1 até n
$\sqrt{}$	raiz quadrada do total

03 · TEORIA · DISTÂNCIAS

Distância euclidiana em NumPy

```python
import numpy as np

a = np.array([1, 2])    # ponto A
b = np.array([4, 6])    # ponto B

distancia = np.sqrt(np.sum((a - b) ** 2))

print(distancia)        # 5.0
```

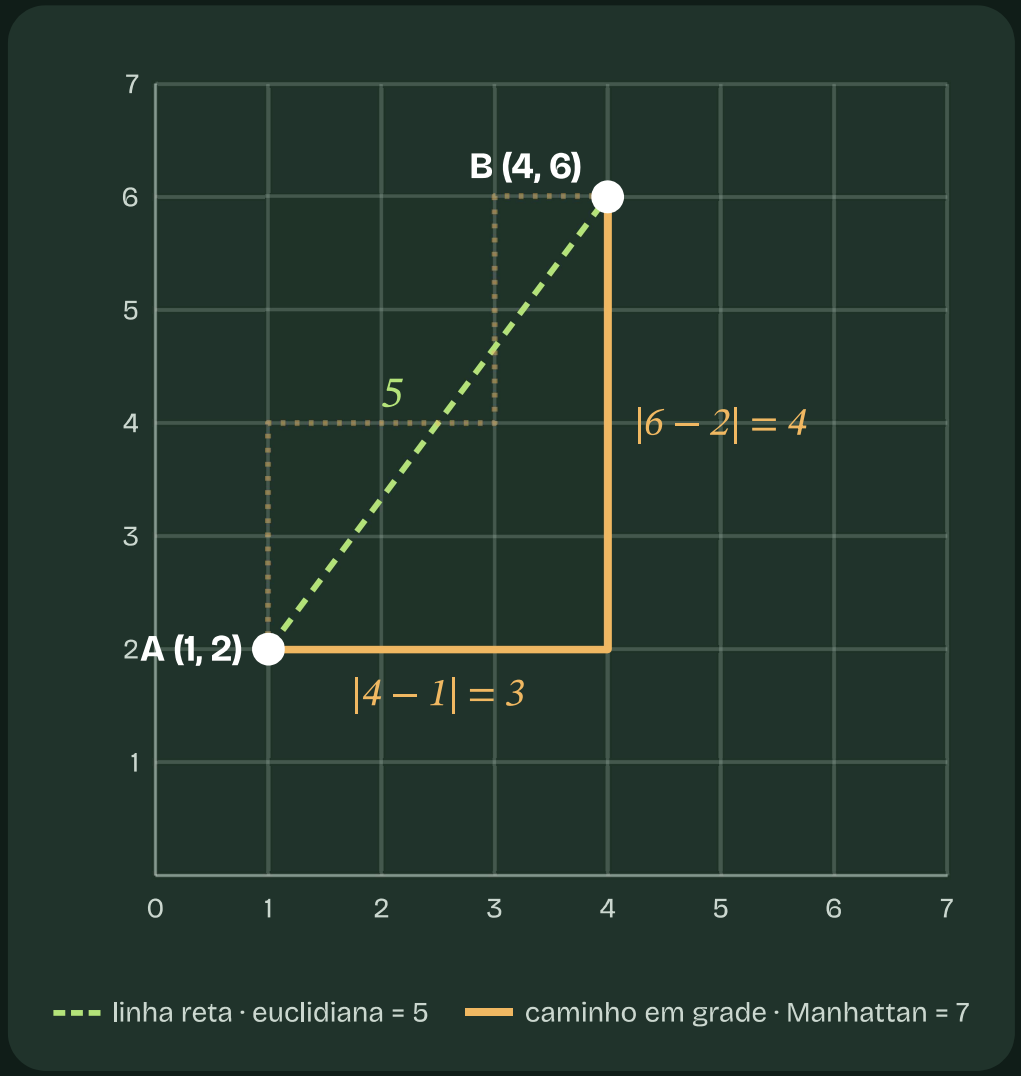
`np.sum(arr)` recebe um array, devolve a soma dos elementos

`np.sqrt(x)` recebe número ou array, devolve a raiz de cada valor

Lendo a linha 6 de dentro para fora

`a - b`	`[-3, -4]`	—
`(a - b) ** 2`	`[9, 16]`	$()^2$
`np.sum(...)`	`25`	Σ
`np.sqrt(...)`	**`5.0`**	$\sqrt{}$

As operações com arrays atuam elemento a elemento: cada posição é uma característica.



03 · TEORIA · DISTÂNCIAS

Distância de Manhattan

O percurso pelas ruas de uma cidade em quadras: só movimentos horizontais e verticais.

$$d(x, y) = \sum_{i=1}^{n} |x_i - y_i|$$

| | valor absoluto: |−3| = 3, |3| = 3

A→B $|4 - 1| + |6 - 2| = 3 + 4 = 7$

```
distancia = np.sum(np.abs(a - b))   # 7
# a - b  ->  [-3, -4]
# np.abs ->  [ 3,  4]
# np.sum ->  7
```

03 · TEORIA · DISTÂNCIAS

Euclidiana vs Manhattan

	Euclidiana	Manhattan
Fórmula	$\sqrt{\Sigma(x_i - y_i)^2}$	$\Sigma\lvert x_i - y_i\rvert$
Operações	diferença, quadrado, soma, raiz	diferença, valor absoluto, soma
Geometria	linha reta	caminho em grade
De A até B	5	7
Pontos à mesma distância	◯ formam um círculo	◇ formam um losango
Uma diferença grande	pesa mais (é elevada ao quadrado)	pesa de forma linear

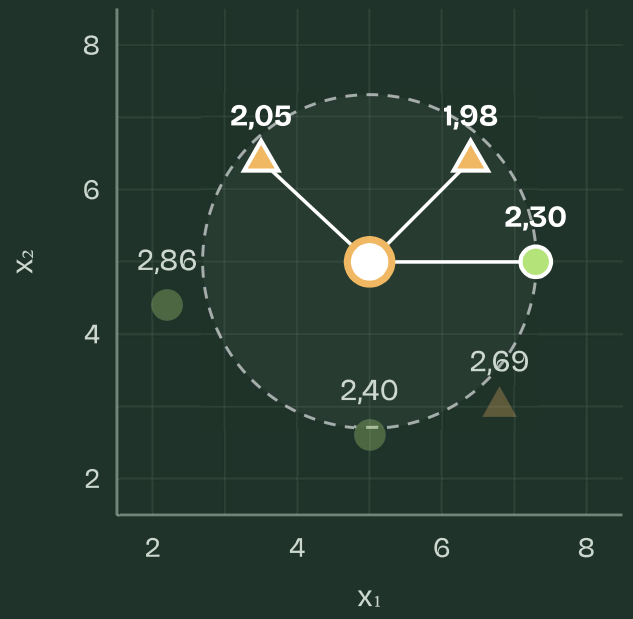
Nenhuma das duas é melhor em todos os casos: a escolha depende dos dados e pode ser validada como um hiperparâmetro.

A métrica pode mudar a classificação

classe 0 · classe 1 · amostra nova

Euclidiana · K = 3

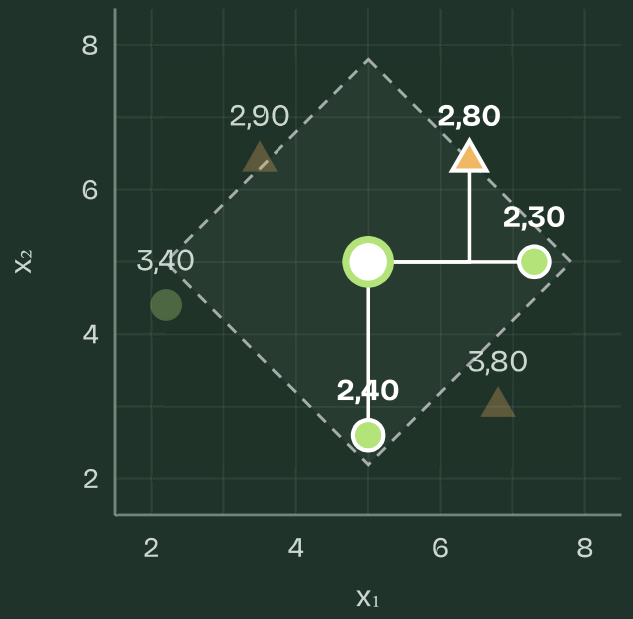
votos [1, 1, 0]



classe prevista **1**

Manhattan · K = 3

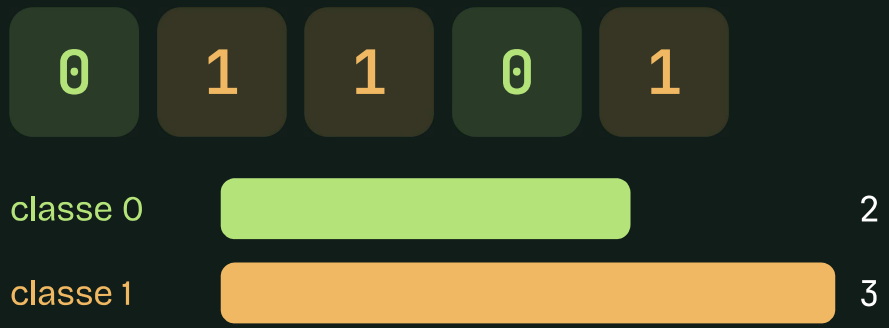
votos [0, 0, 1]



classe prevista **0**

Votação majoritária

Rótulos dos K = 5 vizinhos



Classe prevista **1**

```python
vizinhos = np.array([0, 1, 1, 0, 1])

votos = np.bincount(vizinhos)    # [2, 3]
classe = votos.argmax()           # 1

# alternativa para rótulos quaisquer
classes, contagens = np.unique(
    vizinhos, return_counts=True)
# classes = [0, 1]   contagens = [2, 3]
classe = classes[np.argmax(contagens)]   # 1
```

`np.bincount`	conta cada inteiro 0, 1, 2...; posição c = votos da classe c
`argmax`	índice do maior valor (aqui, a própria classe)
`np.unique`	classes distintas, ordenadas, e suas contagens

03 · TEORIA · VOTAÇÃO

Empates e votação ponderada

Empate

`K = 4 → [0, 1, 1, 0]`

2 votos para cada classe: não há maioria.

K ímpar ajuda, mas não garante

Com 2 classes, K ímpar impede empates.

`3 classes, K = 3 → [0, 1, 2]`

Regra determinística

Nesta aula: em empate, vence o menor rótulo.

`np.bincount(v).argmax()`

Votação ponderada (introdução)

Cada vizinho vota com peso $1 / d$: quem está mais perto pesa mais.

rótulos [1, 0, 0]	uniforme	ponderada
d = [0,5; 2,0; 2,5]	0: 2 · 1: 1	0: 0,9 · 1: 2,0
classe prevista	**0**	**1**

04 · QUIZ RÁPIDO · CÁLCULO À MÃO

Exercício: classifique à mão

Amostra nova **q = (2, 3)**. Calcule no papel, sem código.

ponto	(x_1, x_2)	classe	euclidiana	Manhattan
P1	(1, 1)	0	.....	.....
P2	(4, 3)	1	.....	.....
P3	(2, 5)	1	.....	.....
P4	(3, 2)	0	.....	.....
P5	(0, 4)	0	.....	.....

1 Distância euclidiana de q a cada ponto

2 Distância de Manhattan de q a cada ponto

3 Classe prevista com K = 3, em cada métrica

4 Classe prevista com K = 1. Há empate de distâncias?

04 · QUIZ RÁPIDO · CÁLCULO À MÃO

Resolução do exercício

ponto	(x_1, x_2)	classe	q − P	euclidiana	Manhattan
P1	(1, 1)	**0**	(1, 2)	$\sqrt{5} \approx 2{,}236$	3
P2	(4, 3)	**1**	(-2, 0)	2	2
P3	(2, 5)	**1**	(0, -2)	2	2
P4	(3, 2)	**0**	(-1, 1)	$\sqrt{2} \approx 1{,}414$	2
P5	(0, 4)	**0**	(2, -1)	$\sqrt{5} \approx 2{,}236$	3

Euclidiana

ordem: P4 (1,414), P2 (2), P3 (2)...

K = 3 → rótulos [0, 1, 1] → **classe 1**

K = 1 → P4 → **classe 0**

Manhattan

P2, P3 e P4 empatam em 2

K = 3 → rótulos [1, 1, 0] → **classe 1**

K = 1 → empate; vence o menor índice, P2 → **classe 1**

Empate de distâncias: argsort com kind='stable' mantém a ordem do treino.

04 · QUIZ RÁPIDO

Quiz rápido 1

01

O que significa KNN?

02

O que representa K?

03

O que acontece quando K = 1?

04

Como calculamos a distância entre dois pontos?

05

Qual a diferença entre euclidiana e Manhattan?

06

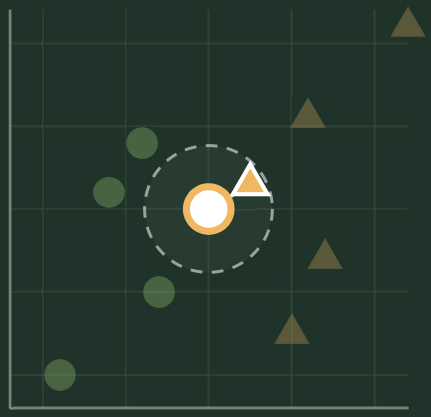
Como o KNN decide a classe? E se houver empate?

03 · TEORIA · ESCOLHA DE K

Como escolher K

classe 0 · classe 1 · amostra nova

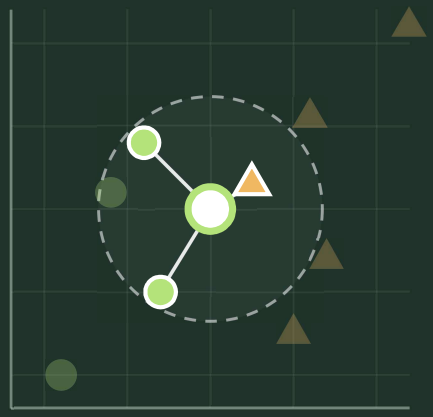
K = 1



[1]

previsão **1**

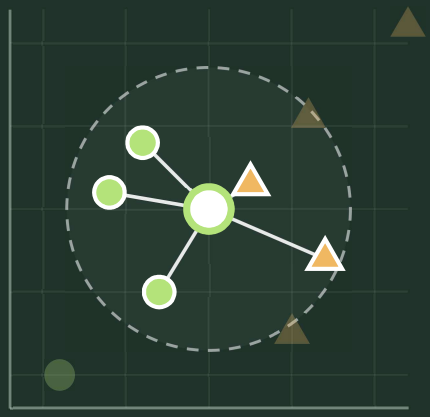
K = 3



[1, 0, 0]

previsão **0**

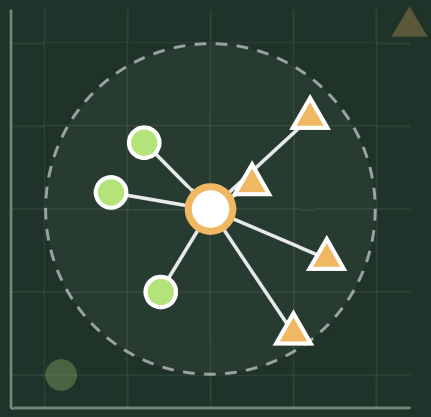
K = 5



[1, 0, 0, 0, 1]

previsão **0**

K = 7



[1, 0, 0, 0, 1, 1, 1]

previsão **1**

K pequeno e K grande

K pequeno

risco: overfitting

poucos vizinhos, decisão local

sensível a ruído e pontos isolados

fronteiras irregulares

K grande

risco: underfitting

muitos vizinhos, decisão suave

pode perder padrões locais

pode favorecer a classe mais frequente

Escolha por validação cruzada (Aula 6.6)

```python
from sklearn.model_selection import KFold
kf = KFold(n_splits=5, shuffle=True, random_state=42)
for k in [1, 3, 5, 7, 9, 11]:
    accs = []
    for tr, va in kf.split(X_train):
        m = KNNDoZero(k=k).fit(X_train[tr], y_train[tr])
        accs.append(m.score(X_train[va], y_train[va]))
    print(k, np.mean(accs))
```

K é inteiro positivo: 1, 2, 3...

K ≤ número de amostras de treino

Escalonamento: o problema das escalas

Cliente novo **Q = (25 anos, R$ 52.000)**. Quem é mais parecido com ele?

cliente	Δ idade	Δ renda	$d^2 = \Delta idade^2 + \Delta renda^2$	distância
A (60, 50.000)	35	2.000	1.225 + 4.000.000	≈ 2.000,3
B (26, 58.000)	1	6.000	1 + 36.000.000	≈ 6.000,0

Composição de d^2 para o cliente A



O KNN escolhe **A**, mesmo com 35 anos de diferença. A característica de maior escala domina a distância.

StandardScaler e MinMaxScaler

`StandardScaler`

$$z = \frac{x - \mu}{\sigma}$$

- μ — média no treino
- σ — desvio padrão no treino

resultado: média 0, desvio 1

`MinMaxScaler`

$$x' = \frac{x - \min}{\max - \min}$$

- min, max — menor e maior valor no treino

resultado: valores entre 0 e 1

Mesmo exemplo, com MinMax

min e max do treino: idade 18 e 70; renda 10 mil e 200 mil

	idade′	renda′
Q	0,135	0,221
A	0,808	0,211
B	0,154	0,253

d(Q, A) ≈ 0,673

d(Q, B) ≈ 0,037

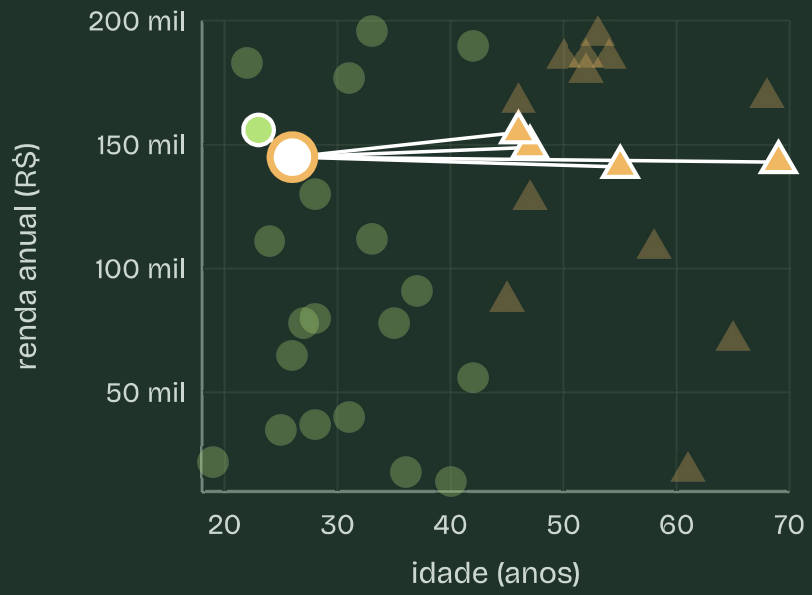
Agora B é o vizinho mais próximo.

Antes e depois do escalonamento

0 · não contratou 1 · contratou cliente novo

Sem escalonamento

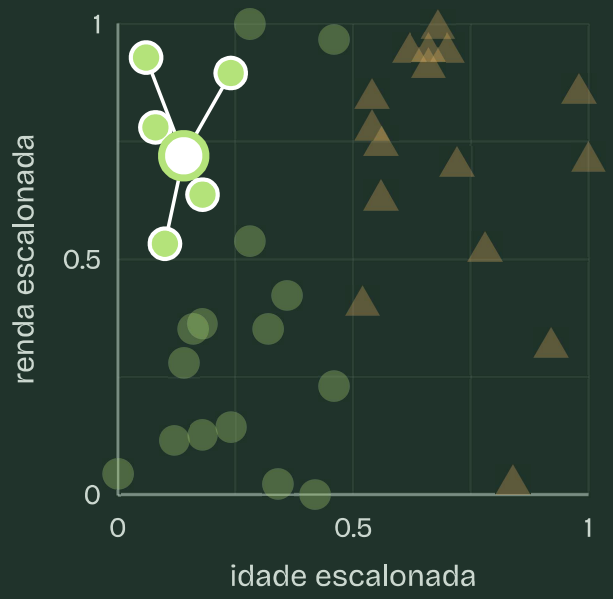
K = 5 · 4 votos classe 1, 1 voto classe 0



vizinhos = renda parecida **1**

Com MinMaxScaler

K = 5 · 5 votos classe 0



vizinhos = idade e renda parecidas **0**

Escalonar sem vazamento de dados

Correto: aprender só no treino

```
X_train, X_test, y_train, y_test = \
    train_test_split(X, y, test_size=0.25,
                     random_state=42)
scaler = StandardScaler()
# fit: aprende μ e σ só no treino
X_train_esc = scaler.fit_transform(X_train)
# transform: reutiliza μ e σ do treino
X_test_esc  = scaler.transform(X_test)
```

Versão manual com NumPy

```
mu    = X_train.mean(axis=0)   # média por coluna
sigma = X_train.std(axis=0)    # desvio por coluna
X_train_esc = (X_train - mu) / sigma
X_test_esc  = (X_test  - mu) / sigma
```

Errado: escalonar antes de dividir

```
X_esc = StandardScaler().fit_transform(X)
X_train, X_test, y_train, y_test = \
    train_test_split(X_esc, y)
# μ e σ incluem o teste: vazamento
```

Errado: escalonador separado no teste

```
X_test_esc = StandardScaler().fit_transform(X_test)
# o mesmo valor bruto vira números diferentes
```

O teste simula dados futuros: ele só pode usar parâmetros aprendidos no treino.

Indexação NumPy

expressão	resultado	significado
`len(X_train)`	`5`	número de linhas (amostras)
`X_train[3]`	`[5, 5]`	linha 3: uma amostra inteira
`X_train[3, 1]`	`5`	linha 3, coluna 1
`X_train[:, 0]`	`[1, 4, 2, 5, 3]`	coluna 0 de todas as linhas
`d[:2]`	`2 primeiros`	fatia início:fim (fim excluído)
`y_train[[1, 3, 4]]`	`[1, 1, 0]`	elementos nas posições da lista
`X_train[y_train == 1]`	`[[4, 6], [5, 5]]`	linhas em que a condição é True
`np.zeros(5)`	`[0., 0., 0., 0., 0.]`	array reservado para resultados

`X_train` · `y_train`

	c0	c1	y
0	1	2	0
1	4	6	1
2	2	1	0
3	5	5	1
4	3	3	0

A posição i é a mesma em X_train e y_train: a amostra i tem rótulo y_train[i].

KNN com loops: distâncias de uma amostra

```python
def distancias_loop(X_train, x):
    distancias = np.zeros(len(X_train))
    for i in range(len(X_train)):
        diff = X_train[i] - x
        distancias[i] = np.sqrt(np.sum(diff ** 2))
    return distancias
```

```python
X_train = np.array([[1, 2], [4, 6], [2, 1],
                    [5, 5], [3, 3]])
y_train = np.array([0, 1, 0, 1, 0])
x = np.array([4, 5])
```

`np.zeros(n)` cria n zeros; `len(X_train)` é o número de amostras de treino.

i	X_train[i]	diff	Σ diff²	distancias[i]
0	[1, 2]	[-3, -3]	18	4.243
1	[4, 6]	[0, 1]	1	1.0
2	[2, 1]	[-2, -4]	20	4.472
3	[5, 5]	[1, 0]	1	1.0
4	[3, 3]	[-1, -2]	5	2.236

KNN com loops: vizinhos e votação

```
k = 3
indices  = np.argsort(distancias, kind='stable')[:k]
vizinhos = y_train[indices]
votos    = np.bincount(vizinhos)
classe   = votos.argmax()
```

`np.argsort(d)`	índices que ordenam d do menor para o maior
`kind='stable'`	em empate, mantém a ordem original
`[:k]`	os k primeiros elementos
`y_train[indices]`	rótulos nas posições escolhidas

```
distancias   [4.243, 1.0, 4.472, 1.0, 2.236]
                    ↓ argsort
ordem                          [1, 3, 4, 0, 2]
              ↓ [:3] → y_train[...]
vizinhos                       [1, 1, 0]
                    ↓ bincount
votos                          [1, 2]
                    ↓ argmax
classe                         1
```

KNN com loops: várias amostras

```python
def predict_loop(X_train, y_train, X_test, k=3):
    previsoes = []
    for x in X_test:                    # loop externo
        distancias = np.zeros(len(X_train))
        for i in range(len(X_train)):   # loop interno
            dif = X_train[i] - x
            distancias[i] = np.sqrt(np.sum(dif ** 2))
        indices = np.argsort(distancias, kind='stable')[:k]
        votos = np.bincount(y_train[indices])
        previsoes.append(votos.argmax())
    return np.array(previsoes)

X_test = np.array([[1, 1], [4, 5], [3, 2]])
predict_loop(X_train, y_train, X_test)    # array([0, 1, 0])
```

x	vizinhos (índices → rótulos)	ŷ
[1, 1]	[0, 2, 4] → [0, 0, 0]	**0**
[4, 5]	[1, 3, 4] → [1, 1, 0]	**1**
[3, 2]	[4, 2, 0] → [0, 0, 0]	**0**

3 × 5 = 15

iterações do loop interno. Em geral: n_test × n_train iterações executadas uma a uma pelo Python.

Dimensões das matrizes

`X_train.shape = (5, 2)`

(n_train, n_features)

col 0	col 1
1	2
4	6
2	1
5	5
3	3

linha 3 = amostra [5, 5]

`X_test.shape = (3, 2)`

(n_test, n_features)

col 0	col 1
1	1
4	5
3	2

`axis=0` percorre as linhas (amostras)

`axis=1` percorre as colunas (características)

```
X_train.shape            # (5, 2)
X_train[3]               # [5, 5]   linha 3
X_train[:, 0]            # [1, 4, 2, 5, 3]   coluna 0
X_train.mean(axis=0)     # [3. , 3.4]  média por coluna
x = np.array([4, 5])     # shape (2,)
x.reshape(1, 2)          # (1, 2): 1 linha, 2 colunas
```

Uma operação com `axis=a` reduz (elimina) o eixo a do resultado.

Loops Python e vetorização

Loop em Python

```
lista = [1.0, 2.0, 3.0, 4.0]
dobro = []
for x in lista:          # 4 iterações
    dobro.append(x * 2)  # interpretadas uma a uma
```

A cada iteração o interpretador lê a instrução, verifica o tipo de x e cria um novo objeto.

Operação vetorizada

```
arr = np.array([1.0, 2.0, 3.0, 4.0])
dobro = arr * 2          # 1 chamada ao NumPy
# [2., 4., 6., 8.]
```

O NumPy guarda os números lado a lado, todos do mesmo tipo, e executa o laço em código compilado.

Vetorização
escrever a operação sobre o array inteiro, sem loop Python

Mesmo resultado
as contas são as mesmas; muda quem executa o laço

Broadcasting
a regra que permite vetorizar entre arrays de shapes diferentes

Broadcasting do zero

1 · array + escalar

```
[1, 2, 3] + 10
```

10 10 10

```
[11, 12, 13]
```

2 · (5, 2) − (2,)

```
[1, 2]   -   [4, 5]   =   [-3, -3]
[4, 6]   -   [4, 5]   =   [ 0, 1]
[2, 1]   -   [4, 5]   =   [-2, -4]
[5, 5]   -   [4, 5]   =   [ 1, 0]
[3, 3]   -   [4, 5]   =   [-1, -2]
```

x é repetido em cada linha: o loop interno inteiro em uma operação.

3 · (3, 1) + (1, 4) → (3, 4)

	0	1	2	3
0	0	1	2	3
10	10	11	12	13
20	20	21	22	23

Coluna e linha são esticadas: todas as combinações.

Regra Alinhe os shapes pela direita. Em cada eixo, os tamanhos devem ser iguais ou um deles deve ser 1 (que é esticado). Eixo ausente conta como 1.

Criando eixos com None

```
A = X_test[:, None, :]      # (3, 2) -> (3, 1, 2)
B = X_train[None, :, :]     # (5, 2) -> (1, 5, 2)
diferencas = A - B          # (3, 5, 2)

# np.newaxis é o mesmo que None
X_test[:, np.newaxis, :].shape   # (3, 1, 2)
```

`:`	todos os elementos deste eixo
`None`	insere um eixo novo, de tamanho 1, nesta posição
`reshape`	alternativa: `X_test.reshape(3, 1, 2)`



Os eixos tracejados de tamanho 1 são esticados: 1 → 5 e 1 → 3. O resultado guarda uma diferença para cada par (teste, treino).

O array de diferenças (3, 5, 2)

`diferencas[i, j, :] = X_test[i] - X_train[j]`

	[0] teste [1, 1]		[1] teste [4, 5]		[2] teste [3, 2]	
treino 0	0	-1	3	3	2	0
treino 1	-3	-5	0	-1	-1	-4
treino 2	-1	0	2	4	1	1
treino 3	-4	-4	-1	0	-2	-3
treino 4	-2	-2	1	2	0	-1

`eixo 0` · camada = amostra de teste `eixo 1` · linha = amostra de treino `eixo 2` · coluna = característica

Matriz de distâncias

```
quadrados  = diferencas ** 2              # (3, 5, 2)
somas      = np.sum(quadrados, axis=2)    # (3, 5)
distancias = np.sqrt(somas)               # (3, 5)

# Manhattan
distancias_man = np.sum(np.abs(diferencas), axis=2)
```

`** 2`	quadrado de cada elemento
`axis=2`	soma as características: o Σ da fórmula
`np.sqrt`	raiz de cada soma

```
somas = [[ 1, 34, 1, 32, 8],
[18, 1, 20, 1, 5],
[ 4, 17, 2, 13, 1]]
```

distancias · shape (n_test, n_train)

	tr 0	tr 1	tr 2	tr 3	tr 4
teste 0	1.0	5.831	1.0	5.657	2.828
teste 1	4.243	1.0	4.472	1.0	2.236
teste 2	2.0	4.123	1.414	3.606	1.0

Elemento [i, j] = distância entre o teste i e o treino j. A linha 1 é igual ao resultado do loop.

Manhattan

```
[[1, 8, 1, 8, 4],
[6, 1, 6, 1, 3],
[2, 5, 2, 5, 1]]
```

Selecionando os K menores valores

índice	0	1	2	3
d	4.5	1.2	3.8	0.9

`np.sort(d)`	`[0.9, 1.2, 3.8, 4.5]`
`np.argsort(d)`	`[3, 1, 2, 0]`
`np.argsort(d)[:2]`	`[3, 1]`
`np.argpartition(d, 1)[:2]`	`[3, 1]` ou `[1, 3]`

sort devolve valores; argsort devolve **posições**. Só as posições levam aos rótulos em `y_train`.

Na matriz de distâncias

```
indices = np.argsort(distancias, axis=1,
                     kind='stable')[:, :k]
# [[0, 2, 4],
#  [1, 3, 4],
#  [4, 2, 0]]

indices = np.argpartition(distancias, k - 1,
                          axis=1)[:, :k]
```

`axis=1` ordena cada linha (cada amostra de teste)

`[:, :k]` todas as linhas, k primeiras colunas

`argpartition` separa os k menores sem ordenar tudo; ordem interna não garantida

A classe KNNDoZero: estrutura

```python
class KNNDoZero:
    def __init__(self, k=3, metrica='euclidiana'):
        if not isinstance(k, (int, np.integer)) or k < 1:
            raise ValueError('k deve ser inteiro positivo')
        if metrica not in ('euclidiana', 'manhattan'):
            raise ValueError('métrica inválida')
        self.k, self.metrica = k, metrica

    def fit(self, X, y):
        self.X_train = np.asarray(X, dtype=float)
        if self.k > len(self.X_train):
            raise ValueError('k > n_train')
        self.classes, self.y_cod = np.unique(
            np.asarray(y), return_inverse=True)
        return self
```

atributo	variável do objeto: `self.k`
método	função da classe: `fit`
`self`	o próprio objeto
`fit`	treinar = validar e armazenar

```
np.unique(y, return_inverse=True)
y = ['limão', 'laranja', 'limão']
classes = ['laranja', 'limão']
y_cod = [1, 0, 1]
```

Votamos com códigos inteiros e aceitamos rótulos numéricos ou texto.

A classe KNNDoZero: predict e score

```python
class KNNDoZero:   # continuação
    def _distancias(self, X):
        dif = X[:, None, :] - self.X_train[None, :, :]
        if self.metrica == 'euclidiana':
            return np.sqrt(np.sum(dif ** 2, axis=2))
        return np.sum(np.abs(dif), axis=2)

    def predict(self, X):
        d = self._distancias(np.asarray(X, dtype=float))
        indices = np.argsort(d, axis=1, kind='stable')
        indices = indices[:, :self.k]
        viz = self.y_cod[indices]
        venc = [np.bincount(v).argmax() for v in viz]
        return self.classes[venc]

    def score(self, X, y):
        return np.mean(self.predict(X) == np.asarray(y))
```

3 testes, 5 treinos, k = 3

d	(3, 5)	distâncias
indices	(3, 3)	[[0,2,4],[1,3,4],[4,2,0]]
viz	(3, 3)	[[0,0,0],[1,1,0],[0,0,0]]
venc	(3,)	[0, 1, 0]
classes[venc]	(3,)	[0, 1, 0]

Empate de votos: vence o menor código, a classe que vem primeiro em `classes`. `score` = média de acertos.

Complexidade e memória

$$O(\text{n_test} \times \text{n_train} \times \text{n_features})$$

cálculo das distâncias; a seleção dos vizinhos soma mais custo

Tempo de execução
segundos medidos

Uso de memória
bytes ocupados pelos arrays

Vetorização
operações em arrays inteiros, em código compilado

Complexidade
como o trabalho cresce; a vetorização não a altera

diferencas com 2.000 testes × 10.000 treinos × 20 características

3,2 GB

400 milhões de float64 × 8 bytes, em um único array intermediário

Processar em blocos

```
def distancias_em_blocos(X_test, X_train, tam=100):
    blocos = []
    for ini in range(0, len(X_test), tam):
        lote = X_test[ini:ini + tam]
        dif = lote[:, None, :] - X_train[None, :, :]
        d = np.sqrt(np.sum(dif ** 2, axis=2))
        blocos.append(d)
    return np.vstack(blocos)   # (n_test, n_train)
```

Com blocos de 100 testes, cada intermediário ocupa 160 MB. Broadcasting é rápido, mas não economiza memória por padrão.

04 · QUIZ RÁPIDO

Quiz rápido 2

01

O que faz `np.argsort()`?

02

O que significa `axis=1`?

03

O que é Broadcasting?

04

Qual a diferença entre loops e operações vetorizadas?

05

Por que Broadcasting pode consumir mais memória?

06

Por que o KNN pode ser lento em grandes datasets?

05 · EXERCÍCIO GUIADO

Exercício guiado em 16 etapas

Notebook `Aula_6_7_Exercicio_Guiado.ipynb` · dataset sintético com semente 42

Dados

01 Criar o dataset

02 Visualizar os pontos

03 Identificar X e y

04 Separar treino e teste

Distâncias

05 Euclidiana à mão

06 Manhattan à mão

07 Função de distâncias

KNN

08 Seleção dos vizinhos

09 Votação

10 KNN com loops

11 KNN com Broadcasting

12 Comparar previsões

Avaliação

13 Medir o tempo

14 Escalonar corretamente

15 Acurácia no teste

16 Fronteiras de decisão

05 · EXERCÍCIO GUIADO

Loop vs Broadcasting

A · Loops

```python
def distancias_loops(X_test, X_train):
    n_test, n_train = len(X_test), len(X_train)
    D = np.zeros((n_test, n_train))
    for i in range(n_test):
        for j in range(n_train):
            D[i, j] = np.sqrt(np.sum((X_test[i] - X_train[j]) ** 2))
    return D
```

B · Broadcasting

```python
def distancias_broadcasting(X_test, X_train):
    dif = X_test[:, None, :] - X_train[None, :, :]
    return np.sqrt(np.sum(dif ** 2, axis=2))
```

Verificação: mesmos vizinhos

```python
D_a = distancias_loops(X_test, X_train)
D_b = distancias_broadcasting(X_test, X_train)
np.allclose(D_a, D_b)                    # True
ia = np.argsort(D_a, axis=1, kind='stable')[:, :k]
ib = np.argsort(D_b, axis=1, kind='stable')[:, :k]
np.array_equal(ia, ib)   # True: mesmos vizinhos
```

05 · EXERCÍCIO GUIADO

Medindo o tempo de execução

```python
from time import perf_counter

def medir(funcao, *args, repeticoes=7):
    tempos = []
    for _ in range(repeticoes):
        inicio = perf_counter()
        funcao(*args)
        fim = perf_counter()
        tempos.append(fim - inicio)
    return np.median(tempos)

t_loops = medir(distancias_loops, X_test, X_train)
t_broad = medir(distancias_broadcasting,
                X_test, X_train)
speedup = t_loops / t_broad
```

$$\text{speedup} = \frac{\texttt{tempo_loops}}{\texttt{tempo_broadcasting}}$$

· várias repetições e a **mediana** reduzem oscilações

· sempre os **mesmos dados** nas duas versões

Resultados medidos pela turma (n_test = 200)

n_train	loops (s)	broad. (s)	speedup
200			
1.000			
5.000			

Os tempos dependem da máquina e são medidos ao vivo; o notebook gera o gráfico de barras com esses valores.

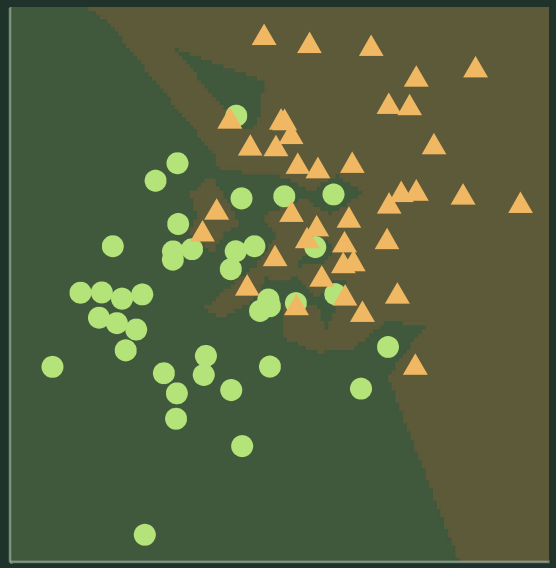
05 · EXERCÍCIO GUIADO

Fronteiras de decisão

● classe 0 · fundo verde = prevê 0 ▲ classe 1 · fundo âmbar = prevê 1

K = 1

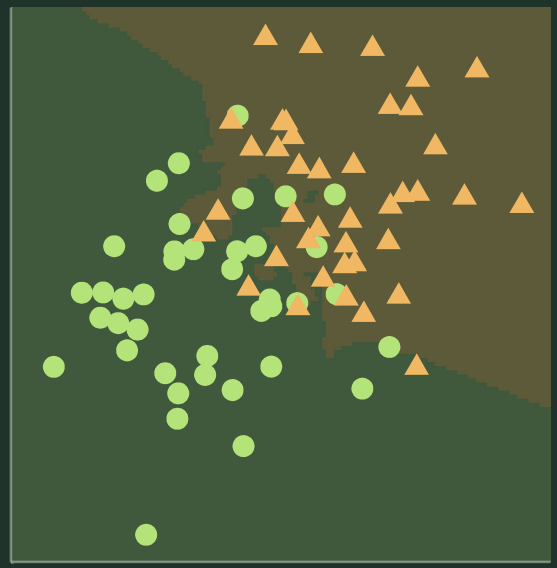
treino: 100%



irregular, com ilhas

K = 3

treino: 88,8%



menos fragmentada

K = 9

treino: 82,5%



regiões suaves

Fundo = previsão do KNN em uma grade de 144 × 136 pontos · 80 exemplos sintéticos, semente 42

Erros comuns: K, distâncias e ordenação

erro e por quê	incorreto	correto
Confundir KNN com K-Fold k do KNN = vizinhos, não folds	`KNNDoZero(k=5) # 5 folds`	`KFold(n_splits=5)` `KNNDoZero(k=3)`
K maior que o treino não existem k vizinhos	`KNNDoZero(k=200)` `.fit(X[:100], y[:100])`	`assert 1 <= k <= len(X_train)`
Esquecer as distâncias ordenar uma coluna não mede proximidade	`np.argsort(X_train[:, 0])[:k]`	`d = distancias_loop(X_train, x)` `np.argsort(d)[:k]`
Valores no lugar de índices sort devolve distâncias, não posições	`y_train[np.sort(d)[:k]]`	`y_train[np.argsort(d)[:k]]`
Ordenação invertida [-k:] pega os mais distantes	`np.argsort(d)[-k:]`	`np.argsort(d)[:k]`

Erros comuns: eixos, shapes e escalas

erro e por quê	incorreto	correto
axis incorreto soma sobre os treinos, não sobre as características	`np.sum(dif ** 2, axis=1)`	`np.sum(dif ** 2, axis=2)`
Shapes incompatíveis (3, 2) – (5, 2): 3 e 5 não combinam	`X_test - X_train`	`X_test[:, None, :]` `- X_train[None, :, :]`
Escalas muito diferentes a renda domina a distância	`modelo.fit(X_idade_renda, y)`	`modelo.fit(X_train_esc, y_train)`
Escalonar teste à parte parâmetros diferentes no treino e no teste	`sc2.fit_transform(X_test)`	`scaler.transform(X_test)`
Escalonar antes de dividir vazamento: μ e σ incluem o teste	`X = sc.fit_transform(X)` `train_test_split(X, y)`	`train_test_split(X, y)` `sc.fit(X_train)`

Erros comuns: votação, tempo e avaliação

erro e por quê	incorreto	correto
Votação incorreta argmax dos rótulos é uma posição	`np.argmax(vizinhos)`	`np.bincount(vizinhos).argmax()`
Ignorar empates resultado muda a cada execução	`rng.choice(empatadas)`	`min(empatadas) # regra fixa`
Tempos com dados diferentes mede o tamanho, não a técnica	`medir(loops, X_test[:50], X_tr)` `medir(broad, X_test, X_tr)`	`medir(loops, X_test, X_tr)` `medir(broad, X_test, X_tr)`
Broadcasting "sem custo" o intermediário pode ter gigabytes	`dif = X_te[:, None] - X_tr[None]`	`n_te * n_tr * n_feat * 8 # bytes` `distancias_em_blocos(...)`
Avaliar só no treino com k = 1, acerta 100% por memorização	`modelo.score(X_train, y_train)`	`modelo.score(X_test, y_test)`

07 · DESAFIO DE CASA

Implemente o KNN do zero e compare loop vs Broadcasting

A implementação deve

- aceitar diferentes K
- distância euclidiana
- distância de Manhattan
- selecionar K vizinhos
- votação majoritária
- gerar previsões
- calcular acurácia
- múltiplas características
- regra explícita e determinística para empates

Versão 1 · Loops

distâncias calculadas percorrendo explicitamente as amostras

Versão 2 · Broadcasting

distâncias calculadas com operações vetorizadas do NumPy

Notebook `Aula_6_7_Desafio_Final.ipynb`, com TODOs

07 · DESAFIO DE CASA

Entregas e critérios de aceitação

Entregas

1 Código das duas implementações

2 Prova de previsões equivalentes

3 Tempos medidos com perf_counter

4 Gráfico comparativo dos tempos

5 Interpretação dos resultados

6 Com e sem escalonamento

7 Fronteiras com diferentes K

8 Conclusão: vantagens e limitações

Critérios de aceitação

✓ O código executa corretamente

✓ Distâncias calculadas corretamente

✓ Vizinhos = menores distâncias

✓ Votação correta

✓ Empates tratados de forma consistente

✓ Escalonamento sem informação do teste

✓ Comparação justa entre as versões

✓ Tempos realmente medidos

✓ Gráficos correspondem às previsões

✓ O estudante explica cada parte

Leitura complementar

NumPy

Broadcasting
numpy.org/doc/stable/user/basics.broadcasting.html

numpy.argsort
numpy.org/doc/stable/reference/generated/numpy.argsort.html

numpy.argpartition
numpy.org/doc/stable/reference/generated/numpy.argpartition.html

Indexação de arrays
numpy.org/doc/stable/user/basics.indexing.html

Arrays: guia rápido
numpy.org/doc/stable/user/quickstart.html

Scikit-learn · Matplotlib · Python

KNeighborsClassifier
scikit-learn.org · sklearn.neighbors.KNeighborsClassifier

StandardScaler
scikit-learn.org · sklearn.preprocessing.StandardScaler

MinMaxScaler
scikit-learn.org · sklearn.preprocessing.MinMaxScaler

Matplotlib: guia do usuário
matplotlib.org/stable/users/index.html

time.perf_counter
docs.python.org/3/library/time.html